Chapter 5 — ML & Deep Learning Foundations

5.1 Tensors and Automatic Differentiation

Conceptual Deep Dive

A tensor is an n-dimensional array with a shape, dtype and device. Neural nets compose differentiable operations; **backpropagation** applies the chain rule backwards through the computation graph: for loss `L` and layer output $y = f(x, w)$, $\partial L/\partial w = \partial L/\partial y \cdot \partial y/\partial w$. Autograd records operations in the forward pass and replays them in reverse. Training loop: forward → loss → `backward()` → optimizer step (SGD, Adam: momentum plus per-parameter adaptive learning rates) → zero grads. Precision: fp32, **bf16** (wide range, preferred for LLM training), fp16 (needs loss scaling), fp8/int8/int4 for inference.

Cross-Connection Scenarios

LoRA (12) trains small matrices with the same machinery; quantisation (12, 13) trades precision for memory.

Production Anti-Patterns

- Forgetting `optimizer.zero_grad()` (gradients accumulate).
- Evaluating without `torch.no_grad()` / `model.eval()` (memory growth, dropout active).
- Mixing devices (CPU tensor with CUDA tensor).

Minimal Working Code

```
import torch

w = torch.tensor([2.0], requires_grad=True)
x, y = torch.tensor([3.0]), torch.tensor([10.0])
opt = torch.optim.SGD([w], lr=0.01)
for _ in range(200):
    opt.zero_grad()
    loss = ((w * x - y) ** 2).mean()
    loss.backward()
    opt.step()
print(w.item())  # approaches 10/3
```

5.2 Neural Network Essentials: Loss, Regularisation, Optimisation

Conceptual Deep Dive

Language models minimise **cross-entropy**: -log P(correct next token). **Perplexity** = exp(mean cross-entropy). Regularisation: dropout, weight decay, early stopping, data augmentation. Normalisation: LayerNorm / RMSNorm stabilise activations. Activations: ReLU, GELU, SwiGLU. Learning-rate schedules: warmup then cosine decay. Gradient clipping prevents explosions. Overfitting shows as train loss falling while validation loss rises.

Cross-Connection Scenarios

Fine-tuning hyperparameters (12); eval metrics (10) include perplexity for base models.

Production Anti-Patterns

- Tuning on the test set.
- No warmup with Adam at high LR.
- Judging a model by train loss alone.

Minimal Working Code

```
import torch
import torch.nn.functional as F

logits = torch.tensor([[2.0, 0.5, -1.0]])
target = torch.tensor([0])
loss = F.cross_entropy(logits, target)
print(loss.item(), "perplexity:", torch.exp(loss).item())

class RMSNorm(torch.nn.Module):
    def __init__(self, d, eps=1e-6):
        super().__init__()
        self.g, self.eps = torch.nn.Parameter(torch.ones(d)), eps
    def forward(self, x):
        return self.g * x * torch.rsqrt(x.pow(2).mean(-1, keepdim=True) + self.eps)
```

5.3 Tokenization: BPE, WordPiece, SentencePiece, tiktoken

Conceptual Deep Dive

Models consume integer token ids. **BPE** (byte-pair encoding) starts from characters/bytes and repeatedly merges the most frequent adjacent pair until reaching a vocabulary size. **Byte-level BPE** (GPT-2, tiktoken) can encode any string with no unknown token. **WordPiece** (BERT) picks merges by likelihood gain and marks continuations with ##. **SentencePiece** treats text as a raw stream (spaces as _), supporting BPE or **Unigram LM**, and is language-agnostic. **tiktoken** is OpenAI's fast BPE implementation. Tokenizer quirks cause real bugs: numbers split unpredictably, whitespace matters, non-English text costs more tokens, glitch tokens, and each model family has its own vocabulary — token counts are not portable.

Cross-Connection Scenarios

Chunk sizes (9) should be measured in the target tokenizer; cost (2.4); fine-tuning needs chat-template-correct special tokens (12).

Production Anti-Patterns

- Counting tokens as len(text)/4 for billing-critical logic.
- Training with a different chat template than inference.
- Assuming one tokenizer for embeddings and generation.

Minimal Working Code

```
from collections import Counter

def train_bpe(corpus: list[str], num_merges: int):
    words = Counter(tuple(w) + ("</w>",) for line in corpus for w in line.split())
    merges = []
    for _ in range(num_merges):
        pairs = Counter()
        for word, freq in words.items():
            for a, b in zip(word, word[1:]):
                pairs[(a, b)] += freq
        if not pairs:
            break
        best = max(pairs, key=pairs.get)
        merges.append(best)
        new_words = Counter()
        for word, freq in words.items():
```

```
            out, i = [], 0
            while i < len(word):
                if i < len(word) - 1 and (word[i], word[i + 1]) == best:
                    out.append(word[i] + word[i + 1]); i += 2
                else:
                    out.append(word[i]); i += 1
            new_words[tuple(out)] += freq
        words = new_words
    return merges

print(train_bpe(["low lower lowest", "new newer newest"], 8))

# Production: import tiktoken; enc = tiktoken.get_encoding("cl100k_base"); len(enc.encode(text))
)
```

5.4 Attention and the Transformer Architecture

Conceptual Deep Dive

Self-attention lets each token mix information from others: Attention(Q,K,V) = softmax(QK^T/√d_k) V. The √d_k scaling keeps dot products from saturating the softmax. **Multi-head** attention runs h smaller attentions in parallel and concatenates, letting heads specialise. Decoder-only LLMs apply a **causal mask** so token i attends only to ≤ i. A transformer block = (norm → attention → residual) + (norm → MLP → residual). Positional information: sinusoidal, learned, **RoPE** (rotary, now dominant), ALiBi. Efficiency variants: **MQA/GQA** (share K/V heads to shrink KV cache), **FlashAttention** (IO-aware exact attention, avoids materialising the n×n matrix), sliding-window attention, Mixture-of-Experts (route each token to a few expert MLPs). Attention cost is O(n^2) in sequence length; that is why long context is expensive.

Architectures: encoder-only (BERT; embeddings/classification), decoder-only (GPT/Llama; generation), encoder-decoder (T5; translation/summarisation).

KV cache: during decoding, past keys/values are stored so each new token needs only its own Q against cached K,V. Cache size ≈ 2 × layers × kv_heads × head_dim × seq_len × bytes × batch; it, not weights, often limits concurrency.

Cross-Connection Scenarios

KV cache and GQA explain vLLM's PagedAttention (13); embeddings models (8) are encoders; context limits shape RAG (9).

Production Anti-Patterns

- Ignoring KV cache memory when sizing GPUs.
- Believing a larger context window means uniform recall (lost-in-the-middle).
- Forgetting the causal mask in custom code (information leak).

Minimal Working Code

```
import math
import torch
import torch.nn as nn

class CausalSelfAttention(nn.Module):
    def __init__(self, d_model: int, n_heads: int):
        super().__init__()
        assert d_model % n_heads == 0
        self.h, self.dk = n_heads, d_model // n_heads
        self.qkv = nn.Linear(d_model, 3 * d_model)
        self.out = nn.Linear(d_model, d_model)

    def forward(self, x):
        B, T, C = x.shape
        q, k, v = self.qkv(x).view(B, T, 3, self.h, self.dk).permute(2, 0, 3, 1, 4)
        att = (q @ k.transpose(-2, -1)) / math.sqrt(self.dk)
```

```
        mask = torch.triu(torch.ones(T, T, dtype=torch.bool, device=x.device), 1)
        att = att.masked_fill(mask, float("-inf")).softmax(-1)
        return self.out((att @ v).transpose(1, 2).reshape(B, T, C))

x = torch.randn(2, 5, 32)
print(CausalSelfAttention(32, 4)(x).shape)

def kv_cache_bytes(layers, kv_heads, head_dim, seq, batch, bytes_per=2):
    return 2 * layers * kv_heads * head_dim * seq * batch * bytes_per
print(kv_cache_bytes(32, 8, 128, 8192, 1) / 1e9, "GB")
```

5.5 Pretraining, Scaling Laws and Post-Training

Conceptual Deep Dive

Pipeline: **pretraining** (next-token prediction on trillions of tokens) → **SFT** (instruction following) → **preference optimisation** (RLHF with PPO, or DPO/ORPO) → increasingly **RL with verifiable rewards** for reasoning models. **Scaling laws** (Kaplan; Chinchilla) say loss falls as a power law in parameters, data and compute; compute-optimal training uses roughly 20 tokens per parameter, though modern models overtrain smaller models for cheaper inference. Reasoning models spend extra tokens at inference ("test-time compute"). Hallucination arises because the objective rewards plausible continuations, not truth.

Cross-Connection Scenarios

Why RAG is needed (9); why evals matter (10); DPO details (12).

Production Anti-Patterns

- Expecting a base model to follow instructions.
- Assuming reasoning models are always better (cost/latency, overthinking).
- Treating the training cutoff as irrelevant.

Minimal Working Code

```
def chinchilla_tokens(params: float) -> float:
    return 20 * params

def training_flops(params: float, tokens: float) -> float:
    return 6 * params * tokens  # standard approximation

p = 7e9
print(f"{chinchilla_tokens(p):.2e} tokens, {training_flops(p, chinchilla_tokens(p)):.2e} FLOPs"
)
```

5.6 Hardware Acceleration: GPUs, TPUs and Memory Math

Conceptual Deep Dive

GPUs run thousands of parallel threads; matrix multiplies map to **tensor cores**. TPUs are matrix-multiply ASICs (systolic arrays) programmed through XLA/JAX. Constraints: **VRAM** (weights + KV cache + activations), **memory bandwidth** (decode is bandwidth-bound: tokens/s ≈ bandwidth / model bytes), **compute** (prefill is compute-bound), and **interconnect** (NVLink, InfiniBand) for multi-GPU. Parallelism: data, tensor, pipeline, expert, and ZeRO/FSDP sharding. Weight memory ≈ params × bytes-per-param (fp16: 2, int8: 1, int4: 0.5). Training with Adam needs ≈ 16 bytes/param without sharding (weights, grads, two moments in higher precision).

Cross-Connection Scenarios

Quantisation formats (12), serving engines (13), cloud GPU choice (13).

Production Anti-Patterns

- Sizing a GPU on weights only.
- Spreading a model across PCIe-connected GPUs and being surprised by latency.
- Leaving GPUs idle at low batch size.

Minimal Working Code

```
def weight_gb(params_b: float, bits: int) -> float:
    return params_b * 1e9 * bits / 8 / 1e9

def decode_tokens_per_s(bandwidth_gbs: float, model_gb: float) -> float:
    return bandwidth_gbs / model_gb

m = weight_gb(70, 4)
print(f"70B @4bit = {m:.0f} GB; ~{decode_tokens_per_s(2000, m):.0f} tok/s upper bound at 2 TB/s"
;)
```

5.7 Sampling and Decoding Strategies in Depth

Conceptual Deep Dive

Greedy decoding picks the argmax each step: deterministic but repetitive. **Beam search** keeps the B best partial sequences by cumulative log-probability (good for translation, poor for open-ended chat). Stochastic methods: temperature, **top-k**, **top-p (nucleus)**, **min-p** (keep tokens with probability at least p times the top token's, adapting to confidence), **typical sampling**, and penalties (repetition, frequency, presence). **Logit bias** nudges or bans tokens; **stop sequences** end generation; **logprobs** expose model confidence for calibration and classification (read the probability of the "yes" token). **Speculative decoding** accelerates without changing the distribution. Edge cases: temperature 0 is not bit-reproducible across hardware/batch sizes; very high top-k with high temperature yields incoherence; repetition penalties can break code and JSON.

Cross-Connection Scenarios

Self-consistency needs T>0 (6.2); classification via logprobs (10.3); constrained decoding masks logits (7.3); speculative decoding (13.1).

Production Anti-Patterns

- Tuning temperature and top-p simultaneously without isolating effects.
- Applying repetition penalties to structured output.
- Treating raw token probabilities as calibrated confidence.

Minimal Working Code

```
import math

def softmax(z, T=1.0):
    m = max(z)
    e = [math.exp((x - m) / T) for x in z]
    s = sum(e)
    return [x / s for x in e]

def min_p_filter(probs, p_base=0.1):
    thr = p_base * max(probs)
    kept = [p if p >= thr else 0.0 for p in probs]
    s = sum(kept)
    return [p / s for p in kept]

def top_k_filter(probs, k):
    cutoff = sorted(probs, reverse=True)[k - 1]
    kept = [p if p >= cutoff else 0.0 for p in probs]
```

```
    s = sum(kept)
    return [p / s for p in kept]

p = softmax([3.0, 2.5, 0.5, -1.0])
print(min_p_filter(p), top_k_filter(p, 2))
```

```
import math

def beam_search(step_fn, start, beam=3, steps=4):
    """step_fn(seq) -> list[(token, logprob)]."""
    beams = [([start], 0.0)]
    for _ in range(steps):
        cand = []
        for seq, score in beams:
            for tok, lp in step_fn(seq):
                cand.append((seq + [tok], score + lp))
        beams = sorted(cand, key=lambda x: -x[1])[:beam]
    return beams

toy = lambda seq: [("a", math.log(.6)), ("b", math.log(.3)), ("c", math.log
(.1))]
print(beam_search(toy, "<s>")[0])
```

5.8 Embedding Models, Encoders and Training Objectives

Conceptual Deep Dive

Encoder models (BERT-family) produce contextual token vectors; pooling (CLS, mean) yields a sentence vector. **Contrastive learning** (InfoNCE) with in-batch negatives and mined **hard negatives** trains retrieval embeddings; **cross-encoders** are trained with pairwise/listwise ranking losses; **distillation** compresses a cross-encoder's judgement into a bi-encoder. Decoder-LLM-based embedders (last-token pooling with instruction prefixes) now top benchmarks. Evaluation: MTEB/BEIR for orientation, your own recall@k for decisions. Edge cases: max sequence length truncation (silent), anisotropy (all vectors similar) fixed by normalisation or whitening, and multilingual mismatch between query and document language.

Cross-Connection Scenarios

Vector search (8), reranking (9.4), fine-tuning embeddings (8.6, 12).

Production Anti-Patterns

- Ignoring max length so long documents embed only their first 512 tokens.
- Choosing a model from MTEB rank without testing on your domain.
- No instruction/prefix when the model requires one.

Minimal Working Code

```
import torch
import torch.nn.functional as F

def info_nce(q: torch.Tensor, d: torch.Tensor, temperature: float = 0.05) -> torch.Tensor:
    """q[i] matches d[i]; all other d[j] in the batch are negatives."""
    q, d = F.normalize(q, dim=-1), F.normalize(d, dim=-1)
    logits = q @ d.T / temperature
    labels = torch.arange(len(q))
    return F.cross_entropy(logits, labels)

torch.manual_seed(0)
q, d = torch.randn(8, 32), torch.randn(8, 32)
print(info_nce(q, d).item())
```

```
import torch

def mean_pool(hidden: torch.Tensor, mask: torch.Tensor) -> torch.Tensor:
    m = mask.unsqueeze(-1).float()
    return (hidden * m).sum(1) / m.sum(1).clamp(min=1e-9)

h = torch.randn(2, 5, 16)
mask = torch.tensor([[1, 1, 1, 0, 0], [1, 1, 1, 1, 1]])
print(mean_pool(h, mask).shape)
```